SPRINTS • AI ENGINEERING PROTOTYPE

Multi-Agent AI Career & Assessment Orchestrator

Comprehensive Technical and Implementation Report (Release v1.1.0)

Status: Hardened Working Prototype	**Orchestration:** LangGraph 1.2+ (StateGraph)
Language: Python 3.10.8	**LLM:** Google Gemini via LangChain
Learner DB: SQLite (Safe Parameterized Ops)	**Sandbox:** Docker (sprints-python-sandbox:latest)
Checkpointer: SqliteSaver (Fernet AES Encrypted)	**Automated CI:** 95 Tests (94 Passing in CI, 1 Skipped)

Project status: Working prototype\
Primary language: Python 3.10.8\
Orchestration: LangGraph\
LLM provider: Google Gemini through LangChain\
Vector database: ChromaDB\
Learner database: SQLite\
Code execution: Docker sandbox\
Test framework: pytest

---

1. Project Overview

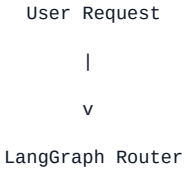
The Sprints Multi-Agent AI Career & Assessment Orchestrator is a Python prototype that routes learner requests to specialized agents. It currently supports two main workflows:

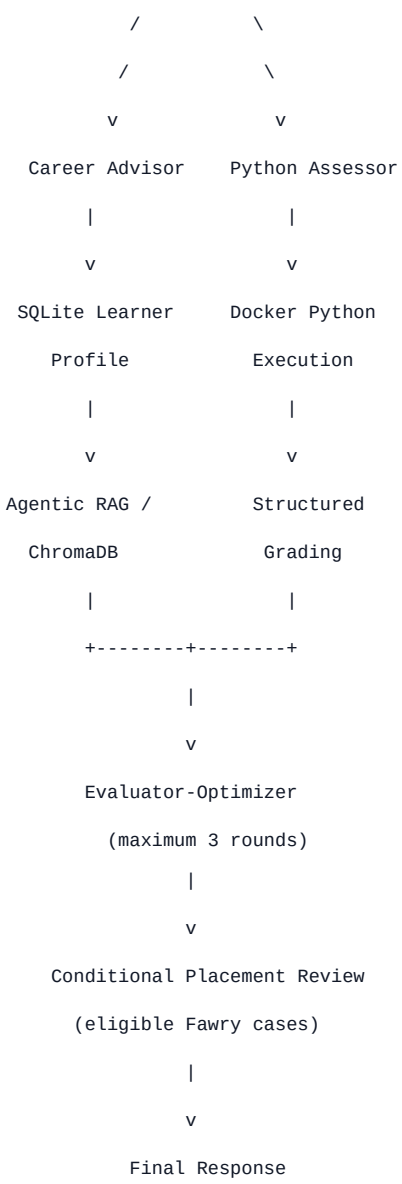
1. **Career Advisor:** retrieves a learner profile from SQLite, searches project knowledge in ChromaDB, evaluates retrieval quality, and generates structured career recommendations.
2. **Python Assessor:** executes submitted Python code in a restricted Docker container and evaluates the result against an assignment.

A central LangGraph workflow coordinates routing, agent execution, response evaluation/optimization, and a conditional human review step for selected placement recommendations.

The project is a prototype. It does not connect to a live jobs or vacancies database, does not contact employers, and does not submit job applications.

2. High-Level Architecture





The graph is implemented with **StateGraph(AgentState)**. The graph builder **build_orchestrator(checkpointer=None)** defaults to LangGraph's **MemorySaver** for fast, isolated in-memory automated testing. For persistent runtime execution, **main.py** (interactive CLI and automated demo) initializes and attaches a persistent **SqliteSaver** checkpointer (writing to **data/orchestrator_checkpoints.sqlite3** with WAL mode, 5-second busy timeout, and thread isolation), guaranteeing restart-safe workflows and durable Human-in-the-Loop review states.

3. Technology Stack

The versions below are taken from the project's active virtual environment using **python -m pip freeze**. Python 3.10.8 is the interpreter version previously reported for this environment.

Component Version

--- ---

Python 3.10.8

LangChain 1.4.7

LangGraph 1.2.18

LangChain Core 1.7.3

LangChain Google GenAI 4.4.2

LangChain Chroma 1.1.0

LangChain HuggingFace 1.2.3

LangChain Text Splitters 1.1.3

ChromaDB 1.5.9

Pydantic 2.14.2

Sentence Transformers 6.2.0

Transformers 5.20.2

PyTorch 2.14.1

NumPy 2.2.6

Pandas 2.3.3

Google GenAI SDK 2.33.0

python-dotenv 1.2.4

python-pptx 1.0.2

pypdf 6.21.4

pytest 9.2.0

The complete environment snapshot is stored in the root **requirements.txt**. It contains all packages from the active environment, including transitive dependencies.

4. Repository Structure

The public repository contains source code, tests, configuration, and documentation. Local datasets, client-provided source materials, learner records, SQLite databases, and ChromaDB persistence files are intentionally excluded.

```
sprints-ai-orchestrator/
■■■ .github/
■   ■■■ workflows/
■       ■■■ tests.yml
■■■ sandbox/
■   ■■■ Dockerfile
■■■ src/
■   ■■■ agents/
■   ■   ■■■ advisor_agent.py
■   ■   ■■■ assessor_agent.py
■   ■   ■■■ evaluator_optimizer.py
■   ■   ■■■ orchestrator.py
■   ■■■ tools/
■   ■   ■■■ db_queries.py
■   ■   ■■■ docker_executor.py
```

```
■   ■   ■■■ rag_retriever.py
■   ■   ■■■ sqlite_checkpoint.py
■   ■   ■■■ test_runner.py
■   ■■■ state.py
■   ■■■ llm.py
■   ■■■ main.py
■■■ docs/
■   ■■■ DEMO_SCRIPT.md
■   ■■■ TECHNICAL_DOCUMENTATION.md
■■■ tests/
■   ■■■ test_agents/
■   ■   ■■■ test_agentic_rag.py
■   ■   ■■■ test_assessor_agent.py
■   ■   ■■■ test_evaluator_optimizer.py
■   ■   ■■■ test_orchestrator_evaluator.py
■   ■■■ test_tools/
■       ■■■ test_db_queries.py
■       ■■■ test_docker_executor.py
■       ■■■ test_sqlite_checkpoint.py
■       ■■■ test_test_runner.py
■■■ LICENSE
■■■ pytest.ini
■■■ requirements.txt
■■■ README.md
■■■ .gitignore
```

The following runtime inputs are expected to be provisioned locally and are not distributed in this public repository:

- Approved curriculum or project-context documents used for ingestion.
- Locally generated processed text and ChromaDB data.
- The SQLite learner database and any learner export files.
- A local **.env** file containing the Gemini API credential.

Do not add client recordings, client presentations, transcripts, learner exports, or identifiable learner records to the public repository. Confirm ownership, consent, and publication permission before sharing any source material.

5. Shared State and Structured Models

src/state.py defines the shared **AgentState** TypedDict used by

LangGraph. It includes fields for:

- Conversation messages and request routing.
- Learner ID and learner profile.
- Retrieved context, source metadata, and retrieval sufficiency.
- Submitted code, assignment, and expected output.
- Grading result and career advice.

- Evaluator feedback and iteration count.
- Human approval requirement, decision, and approval details.
- Errors and final response.

The state module also defines Pydantic models:

`GradingFeedback`

- **score**: integer from 0 to 100.
- **feedback_points**: one or more feedback items.
- **pass_status**: Boolean pass/fail result.

`CareerAdvice`

A structured career response containing the learner name, current track, skill assessment, recommended roles, skill gaps, learning roadmap, salary insight, and overall recommendation.

Using structured output helps keep agent responses aligned with the application's expected data shape.

6. Career Advisor Workflow

The Career Advisor implementation is in **src/agents/advisor_agent.py**.

6.1 Learner profile retrieval

The advisor first calls **get_learner_by_id()** to retrieve the learner record from SQLite. If the ID is missing or no matching record exists, the function returns an **Unable to verify**-style error rather than inventing a profile.

The database track field **bootcamp_track** is normalized into **current_track** for downstream use.

6.2 Agentic RAG

The advisor searches ChromaDB through **retrieve_context()** with a top-k value of 4. Each returned item retains its source and trust-level metadata.

The retrieval process uses three possible attempts: 1. Initial retrieval. 2. First retry after query reformulation. 3. Second retry after another reformulation, if needed.

After each retrieval, a structured retrieval evaluator assesses relevance and evidence sufficiency. If the evidence is insufficient, a query reformulator creates a more focused query based on the missing information. The trace records the query, sources, document count, sufficiency, relevance score, reasoning, and missing information for each attempt.

The maximum is one initial attempt plus two retries (**MAX_RETRIEVAL_RETRIES = 2**).

6.3 Career recommendation generation

When retrieval is sufficient, the advisor passes the verified learner profile and retrieved context to a Gemini model configured for structured **CareerAdvice** output.

The prompt explicitly instructs the model to: - Use the SQLite learner profile as the source of learner facts. - Avoid inventing skills, qualifications, certifications, experience, or assessment results. - Identify skill gaps only when supported by evidence. - Avoid treating project requirements as official employer hiring requirements. - Avoid claiming that an employer has an active vacancy. - Distinguish verified facts from AI-generated recommendations.

The result includes the learner profile, structured advice, sources, retrieval trace, and evidence flags.

7. Python Assessor Workflow

The Python assessment implementation is in **src/agents/assessor_agent.py**.

The assessor receives: - Submitted Python code. - Assignment instructions. - Optional expected output.

The code is executed through **python_repl_tool**, which delegates execution to the Docker executor. If execution fails, the assessor returns an **Unable to verify** error result. If execution succeeds, the assessor uses Gemini structured output matching **GradingFeedback**.

The grading logic checks the submitted solution against the assignment and execution information. The implementation uses a passing threshold of 60 in the assessor.

src/tools/test_runner.py provides an isolated, anti-tampering test runner harness executing deterministic unit test assertions inside the Docker sandbox.

Its security and isolation architecture includes:

- **Single Code Execution:** The learner's code is executed exactly once in the Docker sandbox, capturing standard output and exported variables without redundant re-runs.
- **True OS Process Isolation via **os.fork()**:** Inside the Linux container, each unit assertion case is executed within its own dedicated child process via **os.fork()** and pipe IPC. Memory is isolated via copy-on-write, ensuring complete process boundary separation between individual tests.
- **Anti-Tampering & Exit Trapping:** If student code calls **os._exit()**, **sys.exit()**, or crashes with a segfault, only the child process terminates. The parent harness collects the exit status via **os.waitpid()** and records a structured test failure without aborting the evaluation suite.
- **Pre-Bound Safe Primitives:** Primitives (**_safe_os**, **_safe_sys**, **_safe_json_dumps**) are bound to private references upfront, preventing untrusted code from modifying reporting internals.
- **Dual Assertion & Stdout Verification:** Concurrently evaluates explicit unit assertions (**test_suite**) and expected

standard output (**expected_output**).

• **Unverified State:** Emits **status: "unverified"** when no assertions or expected outputs are provided.

src/agents/assessor_agent.py enforces strict programmatic guardrails:

• **Fail-Closed Safety:** If the automated test runner encounters a technical failure (**test_metrics["status"] == "error"**), the assessor immediately returns **status: "error"** with message **"Unable to verify: Automated test runner execution failed"**, aborting the workflow before invoking the LLM.

• **Deterministic Grade Capping:** If unit test assertions fail (**passed is False**), the score is capped at 50/100, **status: "fail"** is enforced, and failing assertion details are injected into LLM feedback.

src/main.py provides an Interactive CLI (**--mode interactive**) and an Automated Demo (**--mode demo**) driving the central LangGraph Orchestrator. The demo executes both Career Advisor and Python Assessor workflows using synthetic learner profiles (**SYN-DEMO-01**), guaranteeing zero personal data leakage, and handles Human-in-the-Loop review interruptions.

8. Docker Code Execution

The Docker configuration is in **sandbox/Dockerfile**; execution controls are implemented in **src/tools/docker_executor.py**.

The sandbox image is based on **python:3.10-slim**, creates a non-root user with UID 10001, and runs from **/sandbox**.

The runtime configuration includes:

• Network disabled.

• Read-only container root filesystem.

• Memory limit of 128 MB.

• CPU quota of 0.5 CPU.

• Process limit of 64.

• All Linux capabilities dropped.

• **no-new-privileges**.

• A temporary filesystem mounted with **noexec** and **nosuid**.

• A five-second execution timeout.

• Automatic container removal.

• Bounded output capture: stdout and stderr are each limited to 64 KiB.

Output is read in chunks by dedicated reader threads. Once a stream reaches its capture limit, the reader continues draining and discarding additional bytes so a full pipe does not block the child process. The returned result includes an **output_truncated** flag and a truncation marker when applicable. This bounds application memory used for captured output; it is not a substitute for broader host-level disk, logging, or container-runtime controls.

These controls reduce risk but do not establish a production-grade security boundary. Do not expose this prototype to arbitrary public

submissions without independent security review, stronger isolation (e.g. dedicated worker hosts or microVMs), monitoring, and operational limits. Docker daemon access itself is highly privileged and must remain under trusted application control.

9. Evaluator-Optimizer

src/agents/evaluator_optimizer.py evaluates successful agent responses for quality and can request an improved response.

The workflow: 1. Evaluates the original response using a structured evaluation result. 2. If the response does not pass, generates an optimized response using evaluator feedback. 3. Repeats until the evaluator passes or the iteration limit is reached. 4. Preserves the original response if evaluation or optimization fails.

The orchestrator sets **MAX_OPTIMIZATION_ITERATIONS = 3**. The evaluator-optimizer tests use mocks, so they validate control flow without consuming Gemini API quota.

For assessment results, the orchestrator supplies the assignment, expected output, submitted code, execution result, numerical score, and pass status as protected context. The evaluator prompt is instructed not to change execution facts or grading values.

10. LangGraph Orchestration and Human Review

src/agents/orchestrator.py defines and compiles the main graph.

Routing

The router uses the explicit **request_type** first. Otherwise, submitted code plus an assignment routes to the assessor; a learner ID routes to the advisor. Unrecognized requests go to the error node.

Evaluation routing

Successful advisor and assessor results are sent to the evaluator-optimizer. Failed or unverifiable results bypass optimization and end the workflow.

Human-in-the-Loop placement review

The current prototype treats Fawry as a tier-1 employer. A human review is requested only when: - Career advice succeeded. - The verified learner profile lists Fawry as a preferred employer. - The learner's readiness status is **Ready for Placement**.

The placement review node calls LangGraph **interrupt()** with a review payload. The graph can then be resumed with a human decision using **Command(resume=...)** and the same thread/checkpoint context.

The decision records approval, reviewer, comments, learner ID, and employer review. Approval applies only to the recommendation. It does not verify a live vacancy, confirm employer acceptance, contact the employer, or submit an application.

Reviewer Authentication & RBAC Governance:

To prevent unauthorized placement decisions before institutional deployment, **placement_review_node** enforces reviewer authentication against **AUTHORIZED_HITL_REVIEWERS** (configurable via environment variable). If an approval is submitted by an unauthenticated or unauthorized reviewer, governance automatically overrides the decision to rejected (**REJECTED (UNAUTHORIZED REVIEWER)**), logs a security governance alert, and records **reviewer_authenticated: False** in the audit trace.

Configurable Tier-1 Employers:

Qualifying placement review triggers are configurable dynamically via the **TIER_1_EMPLOYERS** environment variable (default: **fawry,vodafone,instapay**), allowing flexible institutional alignment without code modifications.

Persistent Checkpointing

The LangGraph orchestrator employs a dual-checkpointer architecture:

- **MemorySaver** (Default):** **build_orchestrator(checkpointer=None)** defaults to in-memory checkpointing, providing fast, stateless, and fully isolated test runs.
- **SqliteSaver** (Persistent CLI & Demo):** **main.py** explicitly attaches a durable **SqliteSaver** writing to **data/orchestrator_checkpoints.sqlite3**. This provides persistent checkpoint storage across process restarts and supports resuming interrupted Human-in-the-Loop placement reviews.

Concurrency & Thread Safety:

- Enabled SQLite Write-Ahead Logging (**PRAGMA journal_mode = WAL**) and a 5-second busy timeout (**PRAGMA busy_timeout = 5000**) on all database connections to handle concurrent multi-threaded writes.
- Write iterations take atomic snapshots (**list(self.writes[outer_key].items())**) to prevent dictionary mutation errors during concurrent graph node execution.

Security Boundary Note: **SqliteSaver** serializes state using Python's **pickle** module. It is strictly designed for local, trusted filesystem storage. It must never be connected to untrusted SQLite databases or files received from untrusted external sources.

11. Learner Database and Tools

src/tools/db_queries.py provides safe, privacy-hardened SQLite operations.

Removal of Arbitrary SQL Tool:

To eliminate any risk of SQL injection or unauthorized database browsing, the raw **learner_sql_tool** has been removed from the agent-facing tool registry. Agents interact exclusively through safe, parameterized domain tools:

- **get_learner_by_id_tool**: targeted single-learner lookup enforcing profile isolation.
- **search_learners_by_track_tool**: parameterized lookup by track name.
- **get_track_aggregate_stats_tool**: aggregate cohort statistics with minimum cohort suppression (k-anonymity threshold) enforcement.

k-Anonymity Cohort Threshold & Aggregation Privacy:

To prevent individual learner re-identification in sparse tracks, **get_track_aggregate_stats()** enforces

a k-anonymity cohort threshold (**MIN_COHORT_SIZE = 3**). If a requested track contains fewer than 3 enrolled learners, the tool withholds individual aggregate metrics, returning **status: "error"** and **privacy_restricted: True**.

Clarification on Privacy Terminology: This control implements k-anonymity for cohort aggregation. It is distinct from mathematical ε-differential privacy, which requires injecting calibrated Laplacian or Gaussian noise into output distributions.

Zero Arbitrary SQL Functions:

Raw SQL query execution functions (**query_learner_database**, **learner_sql_tool**) have been removed completely from the source code. All database interactions strictly execute via parameterized domain functions with **mode=ro**.

The database is a local runtime dependency and is not included in the public repository. This documentation intentionally omits learner IDs, names, scores, salary targets, employer preferences, and individual readiness details. Use synthetic records for demonstrations and tests.

Do not publish real learner information unless the data owner has explicitly authorized that disclosure and the data has been reviewed for personal or confidential information.

Other tool modules include:

- **document_loader.py**: document ingestion support.
- **audio_transcriber.py**: audio transcription support.
- **rag_retriever.py**: document chunking, embedding, ChromaDB ingestion, and retrieval.
- **code_repl.py**: LangChain tool wrapper for Python execution.
- **docker_executor.py**: Docker-backed execution.

12. Knowledge Sources and Evidence Limitations

Knowledge-base inputs are maintained locally and are not published in this repository. The current ChromaDB collection contains project context, not the complete official Sprints curriculum.

Source documents may contain client or learner information. Before ingestion or redistribution, verify that the material is authorized for the intended use. AI-generated transcripts must be human-reviewed before being treated as authoritative evidence. Incomplete or unverified source material must be clearly labeled and must not be used to assert official curriculum coverage, employer hiring criteria, or current job vacancies.

The learner database does not contain a live job openings table.

13. Installation and Configuration

Prerequisites

- Python 3.10.
- Docker Desktop with the Linux container engine available.
- A Google Gemini API key for live LLM calls.

Create and activate a virtual environment

```
python -m venv .venv
```

```
.\.venv\Scripts\Activate.ps1
```

Install dependencies

```
python -m pip install --upgrade pip
```

```
python -m pip install -r requirements.txt
```

Configure the API key

Create a **.env** file in the repository root:

```
GOOGLE_API_KEY=your_api_key_here
```

Do not commit **.env** or expose the API key in logs, screenshots,

notebooks, or source control. The project loads the key through

python-dotenv.

Build the sandbox image

Run from the repository root:

```
docker build -t sprints-python-sandbox:latest -f sandbox/Dockerfile .
```

Run the current assessor demonstration

```
python -m src.main
```

The current demonstration makes live Gemini calls during grading. If the

API quota is exhausted, it may fail even when Docker execution works.

Automated tests should use mocks and should not require a live API key.

14. Testing

The test suite contains **63 automated tests** across 12 test suites spanning agent workflows, domain tools, logging, and sandbox execution:

- **tests/test_agents/test_agentic_rag.py** (2 tests): validates Agentic RAG retry behavior and sufficiency stopping.
- **tests/test_agents/test_evaluator_optimizer.py** (2 tests): validates iteration limits and optimization convergence.
- **tests/test_agents/test_orchestrator_evaluator.py** (8 tests): validates LangGraph routing, assessment evaluator routing, Human-in-the-Loop interrupt/resume cycles, cryptographic HMAC-SHA256 reviewer token verification, signature tampering rejection, and anti-replay defense.
- **tests/test_agents/test_assessor_agent.py** (4 tests): validates single-pass automated unit test runner integration, execution failure handling, fail-closed runner crash abortion, and deterministic assertion failure score capping.
- **tests/test_agents/test_live_gemini_integration.py** (1 test): validates live Google Gemini API connectivity and structured generation when live credentials are provided.
- **tests/test_tools/test_db_queries.py** (17 tests): validates parameterized single-learner isolation, track searches, max row limits, column allowlisting, SQL injection defense across all domain functions, caller role authorization (**requester_role**), and k-anonymity cohort threshold (**MIN_COHORT_SIZE = 3**) with zero cohort size leakage in error messages.
- **tests/test_tools/test_test_runner.py** (16 tests): validates test parsing, empty code rejection, unverified status emission, combined assertions and stdout execution, full pass and partial failure metrics, compilation error handling, expected stdout matching, **SystemExit** trapping, true OS process isolation (**os.fork()**), delimiter spoofing defense, and 4 dedicated adversarial tests (monkey-patching **os.fork**/**os.waitpid**, **os._exit()** child trapping, builtins corruption immunity, and frame inspection defense).
- **tests/test_tools/test_sqlite_checkpoint.py** (7 tests): validates cross-instance state persistence, thread isolation, 10-thread concurrent writes with WAL mode, simulated process restart resumption, true cross-process persistence spanning separate OS

Python processes, cryptographic HMAC disk tampering detection, and retention policy purging (**purge_old_checkpoints**).

• **tests/test_tools/test_logger.py** (2 tests): validates structured JSON log formatting, telemetry attributes, duration tracking, and event emission.

• **tests/test_tools/test_docker_executor.py** (14 tests): validates bounded 64 KiB streams, empty/invalid submission handling, mock-based timeout and container cleanup, popen exception safety, real container execution, non-zero exit handling, syntax error capture, 5-second runtime timeout enforcement, massive output truncation, read-only root restriction, and network isolation.

Run the complete test suite:

```
pytest -v
```

Continuous Integration is automated via GitHub Actions (**.github/workflows/tests.yml**), running on every push and pull request with 100% pass rate.

15. Known Limitations and Recommended Improvements

Area Current limitation Recommended next

improvement

---------- ---------- ----------

Knowledge base Project context Ingest approved

documents only; not curriculum material and

full official track document versions

curriculum

Source quality Discovery transcript is Human-review and

AI-generated and correct transcript

unverified

Kickoff deck Slides 3 and 4 are Obtain a complete

truncated source deck

Job matching No live vacancy Integrate a trusted

database jobs feed before

claiming vacancy

matches

Placement review Prototype Make employer tiers

Fawry-specific configurable and add

condition reviewer

authentication/audit

Checkpointing **SqliteSaver** local Evaluate distributed

SQLite storage; single checkpointers (Postgres)

node deployment for distributed clusters

Docker execution Prototype sandbox with Evaluate microVM/gVisor

64 KiB output limit isolation and external

and resource bounds; security audit before

not certified for accepting arbitrary

untrusted public code untrusted public code

Test coverage Comprehensive agent & Add live end-to-end API

tool test suite integration benchmarks

(73 tests: 72 passed, 1 skipped in CI)

User interface CLI only (`--mode Build a web dashboard

interactive/demo`) (FastAPI/React) for

students & advisors

Observability Console/logging support Add structured traces,

is basic request IDs, and

privacy-aware logs

---

16. Conclusion

The current implementation demonstrates a multi-agent architecture with

a LangGraph router, a retrieval-grounded career advisor, a Docker-backed

Python assessor, an evaluator-optimizer loop, and a conditional human

approval checkpoint.

The project has a working prototype structure and an extensive passing

test suite (95 automated tests across 12 suites). Its architecture incorporates Master-Worker process isolation,

durable SQLite checkpointing with Fernet AES-128-CBC encryption at rest and coordinate-bound HMAC signatures, PBKDF2 reviewer authentication with brute-force rate limiting, persistent anti-replay nonce storage, and k-anonymity cohort protection. Its current prototype boundaries include the limited project-context

knowledge base and the absence of live vacancy feeds. These

limitations should remain explicit in any demo, report, or presentation.